De C++ al navegador

La tubería de compilación de WebAssembly

Kyungtak · Integración y releases de LLVM, Google Toolchains

JSConf México 2026

Sigue una línea de C++ hasta el interior de un motor de JavaScript.

EL CONTEXTO

Ya estás enviando WebAssembly a producción

Figma

Editor vectorial: C++ en la pestaña

Photoshop Web

El núcleo en C++ de Adobe, compilado

AutoCAD Web

Décadas de C++, en un navegador

Google Earth

Motor 3D como WebAssembly

Juegos en el navegador

Unity / Unreal → WASM

Tus dependencias de npm

Shiki ejecuta Oniguruma como WASM

Apps reales de producción que tus usuarios abren cada día: código compilado corriendo junto a tu JavaScript.

¿Cómo hace una línea de C++ para correr en un navegador de forma segura y rápida?

No «se compila a WASM». Paso a paso: qué le pasa de verdad.

EL PROBLEMA

¿Por qué no solo JavaScript?

Cómputo pesado

Códecs, física, filtros de imagen. Un buen JIT puede igualar un bucle ajustado; no puede prometerlo, y no tiene SIMD.

Un destino de compilación

Otros lenguajes (C++, Rust) necesitaban un lugar donde aterrizar.

Velocidad predecible

Las pausas del GC y los JIT hacen difícil prometer el peor caso.

JavaScript es un pegamento brillante. Necesitábamos un segundo contrato para el trabajo pesado.

DEFINICIÓN

Qué es WebAssembly en realidad

Un formato de instrucciones binario, **portable, aislado y de tipado estático**.

Un destino de compilación, no un lenguaje.

Rara vez se escribe a mano

Es un ensamblador que el motor acepta ejecutar.

Vive junto a tu JS

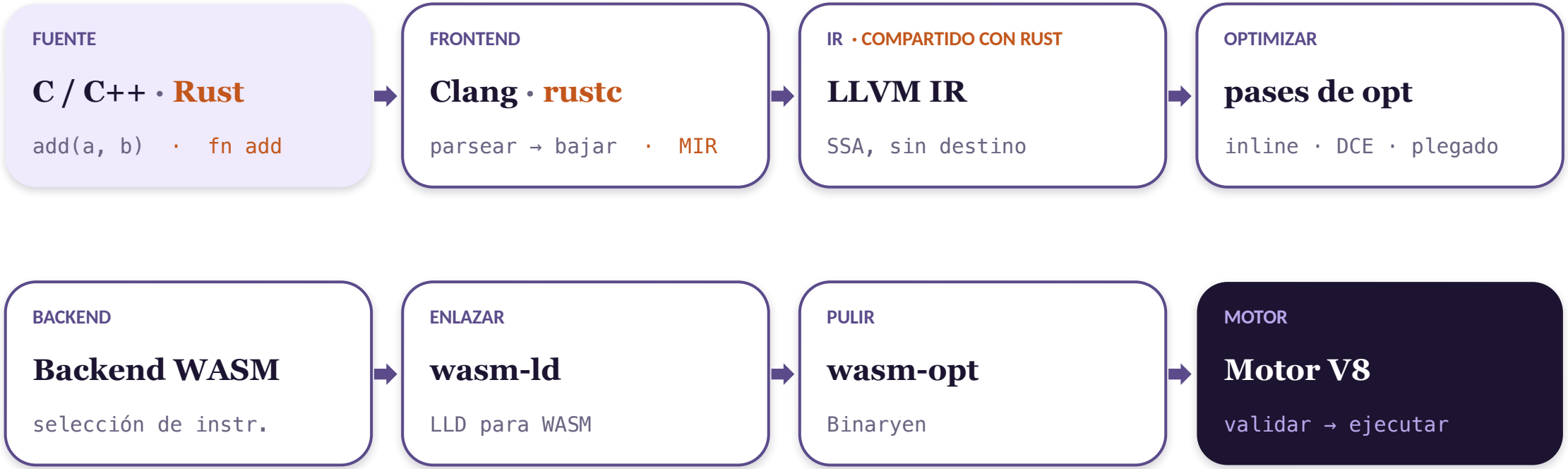
Mismo motor, misma página. JS puede leer su memoria directamente.

«Un CPU con forma de contrato»

De bajo nivel, pero verificable y seguro.

EL MAPA

La tubería completa



El fuente es para humanos. El IR es para el optimizador. El .wasm es el producto que ejecuta el motor.
Rust toma el mismo camino: rustc reemplaza a Clang y baja al mismo LLVM IR; wasm-ld es rust-lld. Del IR en adelante, cada caja es compartida.

NUESTRO ESPÉCIMEN

La línea de código que seguimos

```
int add(int a, int b) {
    return a * 2 + b;
}
```

Trivial a propósito. Mira lo que la tubería le hace.

PASO 1 · FRONTEND

Clang convierte C++ en LLVM IR

FUENTE (para humanos)

```
int add(int a, int b) {
  return a * 2 + b;
}
```

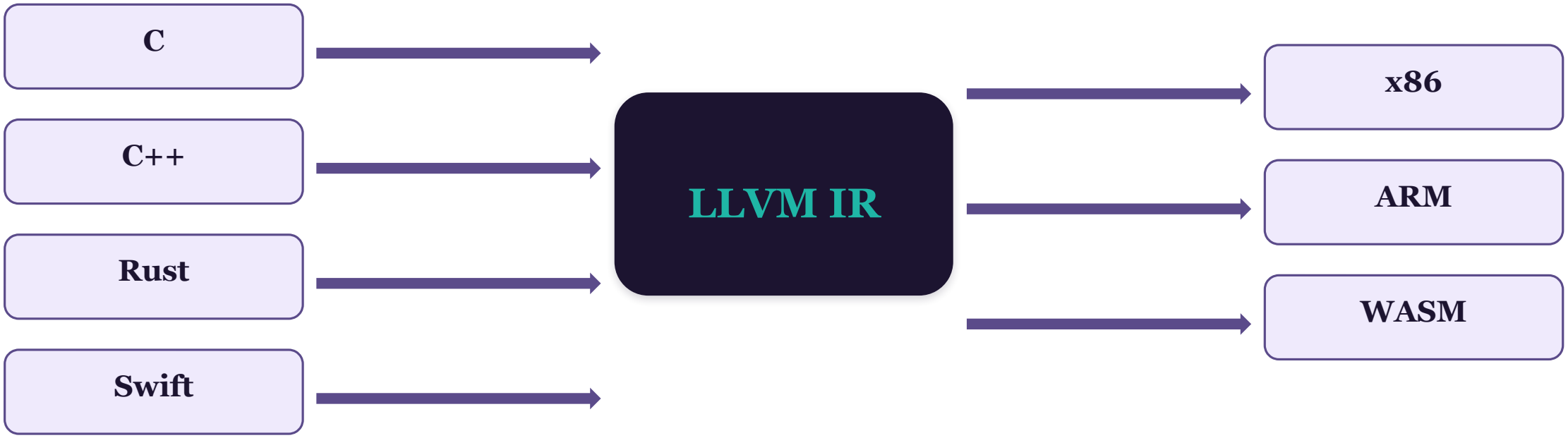
LLVM IR en -O0 (para el optimizador)

```
define i32 @_Z3addii(i32 %0, i32 %1) {
  %3 = alloca i32
  %4 = alloca i32
  store i32 %0, ptr %3
  store i32 %1, ptr %4
  %5 = load i32, ptr %3
  %6 = mul nsw i32 %5, 2
  %7 = load i32, ptr %4
  %8 = add nsw i32 %6, %7
  ret i32 %8
}
```

*En -O0 Clang es literal: las variables viven en memoria y a*2 sigue siendo un mul. (C++ decoró add → _Z3addii.)*

PASO 2 · POR QUÉ IR

El IR es el producto real, no el fuente



M lenguajes × N destinos se vuelve M + N. Ese desacople es lo que hace escalar a todo el ecosistema.

PASO 3 · OPTIMIZAR

La optimización, intuitivamente

Eliminación de código muerto

Borrar cómputos que nadie usa.

```
x = f();  // x nunca se lee
→ eliminado
```

Inlining

Pegar funciones pequeñas dentro de quien las llama.

```
add(2,3) → 2*2+3
→ y luego se pliega a 7
```

Plegado de constantes

Calcular constantes al compilar.

```
2 * 21
→ 42
```

LOS PASES QUE HICIERON EL TRABAJO EN ESTA CHARLA

`mem2reg / SROA`	las locales se vuelven valores SSA: el 9 → 3 de la diapositiva 11
`instcombine`	álgebra por instrucción: mul 2 → shl 1
`GVN / CSE`	calcular una vez, reusar el valor
`LICM`	cuerpos desenrollados, menos saltos (el hash)
`loop unroll`	bucles escalares a carriles SIMD, cuando puede
`loop / SLP vectorize`	llamada virtual → directa cuando se conoce el tipo
`desvirtualización`	constantes y cuerpos fluyen entre funciones
`IPSCCP / inlining`	

Pases sobre el IR. Mismo significado, menos máquina. -O2 / -O3 / -Oz eligen los preajustes.

PASO 3 · NUESTRO ESPÉCIMEN

Lo que -O2 le hizo a add()

LLVM IR en -O2

```
define i32 @_Z3addii(i32 %0, i32 %1) {
  %3 = shl nsw i32 %0, 1
  %4 = add nsw i32 %3, %1
  ret i32 %4
}
```

QUÉ CAMBIÓ

```
alloca / store / load  →  desaparecen
  mem2reg: los valores viven en registros

mul %a, 2  →  shl %a, 1
  instcombine: un shift es más barato

9 instrucciones  →  3
```

*Mismo significado, un tercio del trabajo. El truco a*2 → shift es del optimizador, no del frontend.*

PASO 4 · EMITIR + ENLAZAR

Del IR a un binario .wasm

1	`Backend WASM`	Selección de instrucciones: IR → ops de WebAssembly.
2	`wasm-ld`	El enlazador de LLVM (LLD) para WASM: resuelve símbolos, distribuye la memoria.
3	`.wasm`	Un artefacto binario real y portable.
4	`wasm-opt`	Binaryen pule el módulo entero en tamaño y velocidad.

Emscripten orquesta todo esto y añade el pegamento JS: runtime, shims de libc, sistema de archivos.

PASO 4 · PULIR

wasm-opt: lo que hace Binaryen

Locales fusionadas

El backend de LLVM gasta una local por valor. Binaryen las reusa y deja los valores en la pila.

```
local.set 2
local.get 2
i32.add
→ i32.add, sin local
```

Peepholes sobre ops WASM

Reescrituras que LLVM nunca ve, porque esas formas solo existen tras la selección de instrucciones.

```
if c (then 1)
     (else 0)
→ c
  (c ya es 0 o 1)
```

Vista de todo el módulo

Todas las funciones a la vez: los cuerpos idénticos se fusionan, y lo que ningún export alcanza desaparece.

```
func $f … func $g
  mismos bytes
→ una sola función
→ inalcanzable: fuera
```

Empaquetado de memoria

Los segmentos de datos llenos de ceros desaparecen. La memoria lineal empieza en cero de todos modos.

```
(data "\00 … \00 \01")
→ (data "\01") al final
→ 60 KB menos por la red
```

POR QUÉ AQUÍ NO CAMBIÓ NADA · MEDIDO CON BINARYEN 133 EN ESTA DEMO

Módulo -O3: 2,374 → 2,383 bytes, 558 → 558 instrucciones. Solo cambiaron los números de las locales; los 9 bytes extra son una sección de metadatos.
LLVM ya tenía el programa completo: un solo archivo fuente, sin libc, exports explícitos. Inlining, DCE y las locales quedaron resueltos en -O3.
Módulo -O0: 3,432 → 2,644 bytes, 23% más chico. wasm-opt rehace buena parte del trabajo del optimizador a posteriori, pero no todo: -O3 da 2,374.
Una app real con Emscripten: docenas de objetos, libc, pegamento de runtime. Ahí la vista de todo el módulo suele quitar entre 10 y 20%.

PASO 4 · NUESTRO ESPÉCIMEN

add() como WebAssembly

TEXTO WASM (wasm2wat add.wasm)

```
(func $add (param i32 i32) (result i32)
  local.get 0
  i32.const 1
  i32.shl
  local.get 1
  i32.add)
```

LÉELO COMO UNA PILA

```
push a
push 1
shift left      →  a*2
push b
add             →  a*2 + b
```

WASM es una máquina de pila. El archivo .wasm completo: unos 200 bytes, 14 de ellos código. La cuenta exacta depende de qué Clang lo construyó.

PASO 5 · EL MOTOR

Dentro del motor de JavaScript

Validar

Una verificación de tipos en una sola pasada sobre todo el módulo, antes de que nada se ejecute. Un módulo que falla nunca corre. El paso más barato y el más importante: es lo que permite a un navegador confiar en un binario de un desconocido.

Compilar, dos veces

Liftoff compila cada función rápido, con código mediocre, para que la página arranque ya. Luego V8 observa qué funciones están calientes y recompila solo esas con TurboFan, en un hilo de fondo. El primer render tras cambiar de build en la demo es código de Liftoff.

Memoria lineal

El módulo recibe un ArrayBuffer y nada más. Un puntero es solo un índice dentro de él. Cada load y store se verifica contra los límites, así que el módulo no puede tocar nada fuera de su propio arreglo.

La frontera

Solo los números cruzan directamente entre JavaScript y el módulo: i32, i64, f32, f64. Todo lo demás son bytes en ese arreglo. La siguiente diapositiva muestra lo que eso cuesta.

Validar → Liftoff → el código caliente sube de nivel en segundo plano → ejecutar. Seguro por construcción: tipos, límites, firmas.

LA FRONTERA

Cruzar JS ↔ WASM

TU LADO (JavaScript)

```
// 1  fetch, validar, compilar: una llamada
const { instance } = await WebAssembly
  .instantiateStreaming(fetch("add.wasm"));

// 2  llamar un export: entran números, salen números
instance.exports.add(20, 2);          // 42

// 3  mirar su memoria: una vista, no una copia
const { memory } = instance.exports;
const px = new Uint8ClampedArray(
  memory.buffer, ptr, w * h * 4);
```

LO QUE MUERDE, Y DÓNDE LO VERÁS

Solo cruzan números

i32, i64, f32, f64. Un i64 llega como BigInt. Pestaña 3, tarjeta 2.

Strings y arreglos son puntero + longitud

Codifica a bytes, escríbelos en la memoria, pasa el índice y la cuenta. Pestaña 3, tarjeta 3.

memory.grow() desconecta las vistas viejas

El ArrayBuffer se reemplaza; vuelve a crear tus typed arrays. Pestaña 3, tarjeta 5.

Cruza una vez, trabaja mucho

Cada llamada tiene un costo fijo. Entrega un cuadro, no un píxel.

Emscripten y wasm-bindgen escriben este pegamento por ti. Saber qué hace es cómo conservas la velocidad que viniste a buscar.

POR QUÉ FUNCIONA

Cuatro palabras, desmitificadas

Máquina de pila

Sin registros. Cada instrucción saca sus entradas de una pila tipada y mete su resultado: push a, push 1, shift, push b, add. Tan simple que el motor verifica cada instrucción en una pasada antes de ejecutar ninguna.

Memoria lineal

Todo el mundo del módulo es un arreglo plano y creciente de bytes. Un puntero de C++ es solo un índice dentro de él. Cada load y store se verifica contra los límites, y JavaScript ve los mismos bytes como un ArrayBuffer.

Caja de arena

Nada es ambiental. Sin DOM, sin archivos, sin red, sin otra memoria. El módulo solo puede llamar a las funciones que JavaScript le entregó, y solo corre cuando la validación dice que cada instrucción está bien tipada.

Binario portable

Un formato para cada CPU y cada motor. La cadena de herramientas decide qué dicen los bytes; el motor solo decide qué tan rápido corren. Los mismos bytes dan el mismo resultado en cualquier máquina.

Verificado una vez, encerrado en un arreglo, sin salida salvo los imports, los mismos bytes en todas partes: por eso un navegador puede correr un binario que nunca ha visto.

DEMO EN VIVO

Veámoslo correr

1 · La tubería, en vivo

add() → una plantilla que desaparece → constexpr → una llamada virtual. Fuente, IR en -O0, IR en -O2, texto WASM y los bytes, para cada uno.

2 · El optimizador, medido

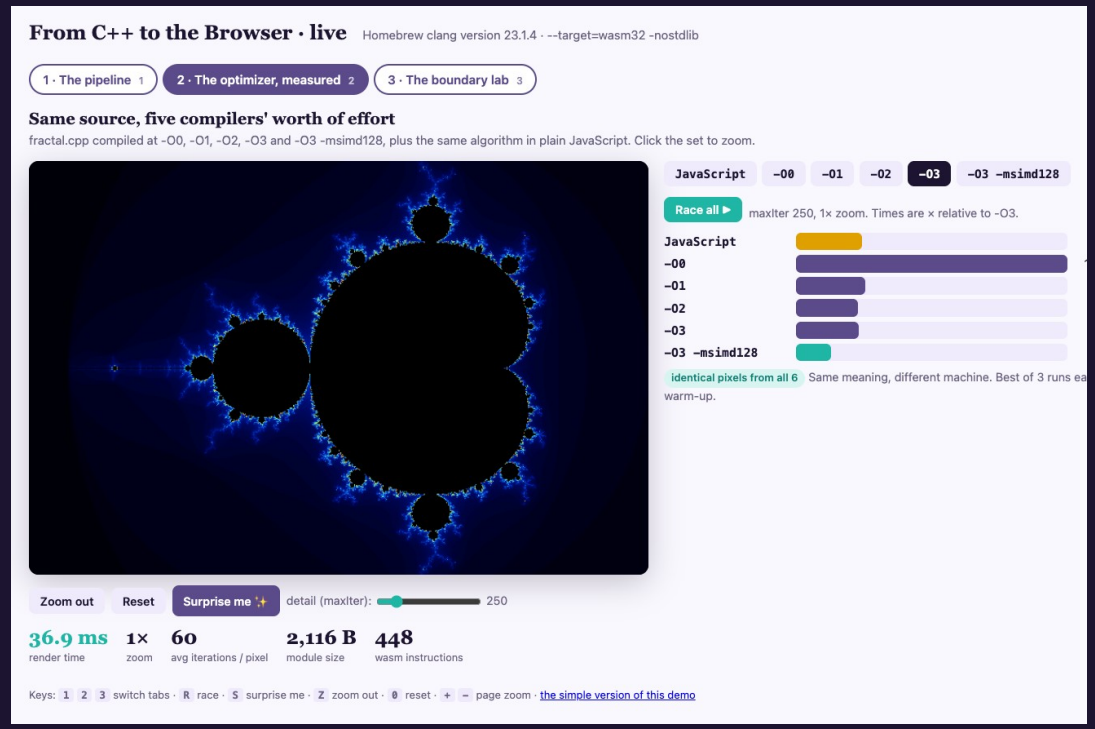
JavaScript contra el mismo C++ en -O0, -O1, -O2, -O3 y -O3 con SIMD. Mismo fuente, mismo motor, píxeles idénticos.

3 · El laboratorio de la frontera

add(20, 2) · un i64 llega como BigInt · un string es puntero + longitud · memory.grow() desconecta todas las vistas.

4 · Donde JavaScript no puede empatar

Un hash de 64 bits de cuatro maneras: BigInt, mitades de 32 bits a mano, C++ en -O0 y -O3. El mismo hash en los cuatro; los tiempos no.



La pestaña de la carrera: seis builds de un archivo C++, píxeles idénticos, los milisegundos cuentan la historia.

`kevinwkt.github.io/from-cpp-to-the-browser/demo` `teclas: 1 2 3 4 · R carrera (2 o 4) · S sorpresa · Z alejar · 0 reset`

ACTO II

La sala de máquinas

Todo hasta ahora fue el camino feliz.

Ahora: lo que cuesta operar esta tubería para miles de ingenieros.

MI TRABAJO DIARIO

Tomo el LLVM de upstream y lo aterrizo dentro de Google.

Integración

Construir el compilador de código abierto contra una base de código interna gigante.

Trenes de release

Mover a todos a un compilador más nuevo sin romperle nada a nadie.

Reproducibilidad

Mismas entradas → mismos bits, en cada máquina, para siempre.

Acceso de commit en upstream

Arreglar los problemas en el origen y devolverlos.

HISTORIA DE GUERRA

Una actualización «correcta» del compilador. Un puñado de binarios sutilmente mal.

1. **Todo en verde** — Todas las pruebas pasaron. La actualización se veía completamente limpia.
2. **Equivocado con confianza** — No un crash: unos pocos binarios producían una salida sutilmente incorrecta.
3. **Bisecar a escala** — Acotar un rango de cambios de upstream a través de muchos destinos.
4. **El punto: confianza** — A escala, «compila» no es la vara. Lo es una corrección en la que puedas confiar.

A ESCALA

La reproducibilidad convierte una herramienta en infraestructura

Mismas entradas → los mismos bits, cada vez.

Builds herméticos

Entradas selladas; nada se filtra desde la máquina.

Toolchains fijadas

Todos en una versión exacta y conocida del compilador.

Bit por bit

Dos builds del mismo código son idénticos byte a byte.

Es lo que permite a miles de ingenieros confiar en la toolchain lo suficiente como para nunca pensar en ella.

SEAMOS HONESTOS

Los bordes filosos

Depurar es más difícil

DWARF en wasm y los source maps existen, pero van detrás de lo nativo.

La frontera cuesta

Cruzar JS ↔ WASM y copiar datos no es gratis.

Las herramientas siguen cambiando

La especificación se mueve rápido; lo «estable» es reciente, no antiguo.

No siempre gana

Para trabajo ligero con el DOM, JS a secas es más simple y a menudo más rápido.

Los diseños parlanchines entre JS y WASM pierden justo la velocidad que viniste a buscar. El encaje importa más que una tabla de posiciones.

HACIA DÓNDE VA

WASM crece más allá del navegador

WASI 0.2: estable	Enero 2024. Más el Component Model: interoperabilidad tipada entre lenguajes.
WASI 0.3: async	Junio 2026. Async nativo con futures y streams; en Wasmtime y jco.
Servidor y edge	Wasmtime, Node, Deno y las plataformas edge corren los mismos binarios.

Y Wasm 3.0 (sep 2025) añadió GC, así que Kotlin, Dart y Java ahora también compilan a él.

Empezamos con una línea de C++.

Bajada, optimizada, emitida, enlazada, pulida, validada, compilada dos veces, y ahora corriendo en una caja de arena en tu navegador.

El fuente nunca fue el producto.
La toolchain es el producto.

Gracias.

Hablemos de compiladores y toolchains.

Kyungtak

Integración y releases de LLVM, Google Toolchains

Repo + diapositivas: github.com/kevinwkt/from-cpp-to-the-browser

Demo en vivo: kevinwkt.github.io/from-cpp-to-the-browser/demo

La grabación se publica después de la charla.